

SPRINT 01

Technical Report

Industrial Equipment Monitoring Platform

Document	Sprint 01 Technical Report
Milestone	M1 - Contracts and Vertical Slice
Role	Technical Lead
Baseline	Requirements Specification v1.0 - Draft baseline
Release	Portfolio MVP
Report date	15 September 2026

Purpose: define and freeze the technical contracts needed before gateway reliability, alert workflow, and dashboard integration are implemented.

Document Control

Source basis

This report is derived from the Industrial Equipment Monitoring Platform Requirements Specification v1.0 (Draft baseline) and expands the Sprint 01 Technical Lead interpretation of M1 without changing the requirements baseline.

Item	Value	Notes
Sprint	01	Contract-first technical foundation
Milestone	M1	Requirements, telemetry schema, migrations, simulator-to-API ingestion
Primary focus	Telemetry semantics	Identity, timestamps, idempotency, partial batches, ordering
Deferred implementation	M3 / M4 items	Gateway reliability and full alert workflow are not Sprint 01 deliverables

Report Contents

- 1. Sprint Goal and Scope
- 2. Sprint 01 Vertical Slice
- 3. Event Identity and Idempotency
- 4. Timestamp Model
- 5. Partial Batch Outcomes and Transaction Rules
- 6. Gateway Queue Lifecycle
- 7. Alert Ordering and State Model
- 8. Site and Authorization Boundary
- 9. Technical Decision Register
- 10. Open Questions
- 11. Sprint Exit Criteria and Definition of Done
- Appendix A. API Contract Examples

1. Sprint Goal and Scope

Sprint 01 should be treated as a contract-first sprint, not as a feature-count sprint. M1 exists to stabilize the telemetry contract and prove a working simulator-to-API path before later work depends on those semantics.

Sprint Goal

Establish a stable, versioned telemetry contract and prove that simulated equipment readings can be ingested idempotently, validated individually, persisted correctly, and interpreted consistently by future gateway, history, and alert components.

1.1 Scope committed for Sprint 01

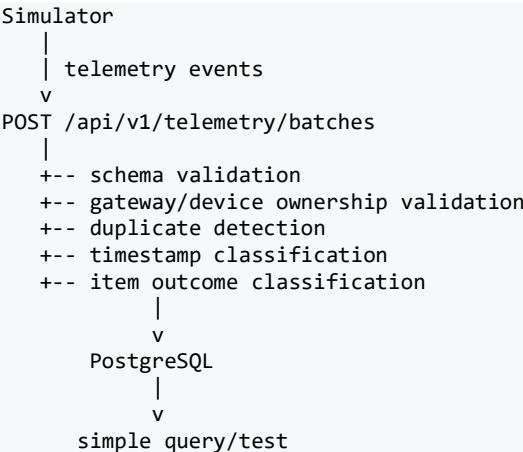
Area	Sprint 01 deliverable
Telemetry contract	Versioned request and response schema.
Event identity	Final definition of event uniqueness and retry identity.
Timestamp semantics	Ownership and use of device, gateway, and backend timestamps.
Database	Minimum device and telemetry schema plus migrations.
Ingestion	Batch endpoint for simulator-to-API ingestion.
Idempotency	Duplicate detection and identity-conflict handling.
Partial batches	Per-item outcomes so valid records can succeed in a mixed batch.
Simulator	Deterministic valid, invalid, duplicate, and out-of-order events.
Current-reading semantics	Define latest-value protection so older replay cannot replace newer current state.
Alert ordering contract	Freeze ordering semantics now; implement full alert workflow later.
Automated tests	Contract, duplicate, mixed-batch, transaction, and ordering tests.
Documentation	Decision log, ADR candidates, API examples, and unresolved questions.

1.2 Explicitly deferred from Sprint 01

- Full persistent gateway buffering/retry/restart implementation (primarily M3 - Reliability).
- Complete authentication, site-permission workflow, connection-status workflow, and alert workflow (primarily M4 - Operational Workflow).
- Production dashboard features beyond what is needed to verify the vertical slice.

2. Sprint 01 Vertical Slice

The vertical slice should be intentionally narrow. It proves the contract from a deterministic simulator through backend validation and persistence, with enough query/test support to verify outcomes.



Technical Lead position

A successful Sprint 01 is not measured by UI completeness. It is measured by deterministic contract behavior that later gateway and alert work can rely on without schema redesign.

3. Event Identity and Idempotency

The requirements already define a stable retry identity: retries preserve device ID, boot ID, and sequence number; a new device boot receives a new boot ID. The database must enforce uniqueness on the same tuple.

ADR-001 - Telemetry Event Identity

Event identity is the composite key (device_id, boot_id, sequence_number). A separate random event UUID is not required for the MVP.

```
event_identity =
  device_id
  + boot_id
  + sequence_number
```

3.1 Example identity

```
{
  "device_id": "motor-001",
  "boot_id": "7a9678d2-3319-47e2-a5be-8c688978720b",
  "sequence_number": 1842
}
```

Existing record	Incoming event	Backend result
No	First occurrence	accepted
Yes - same identity and same immutable content	Retry	duplicate
Yes - same identity but different immutable content	Identity reuse with changed data	rejected_identity_conflict

3.2 Immutable telemetry content

For duplicate comparison, gateway and backend receipt timestamps should not be treated as device content. The immutable event content should include:

```
schema_version
device_id
boot_id
sequence_number
measured_at
device_uptime_ms
metric
value
unit
quality

same identity + same immutable telemetry
=> duplicate

same identity + different immutable telemetry
=> identity conflict
```

PostgreSQL may use a surrogate internal primary key, but the domain uniqueness constraint remains:

```
UNIQUE(device_id, boot_id, sequence_number)
```

4. Timestamp Model

The system has multiple clocks. They must remain semantically distinct so replay, history, freshness, and alert decisions do not accidentally use backend arrival time as measurement time.

ADR-002 - Timestamp Ownership

The device owns `measured_at` and `device_uptime_ms`; the gateway owns `gateway_received_at`; the backend owns `backend_received_at`. All stored absolute timestamps use UTC.

Timestamp	Owner	Meaning	Caller override?
<code>measured_at</code>	Device	Physical measurement time when clock is trustworthy	Device payload value
<code>device_uptime_ms</code>	Device	Elapsed time since current boot	Device payload value
<code>gateway_received_at</code>	Gateway	Original receipt time at gateway	Not ESP32 supplied
<code>backend_received_at</code>	Backend	Backend processing/receipt time	Never client supplied

4.1 Replay example

```

10:00:00  sensor measures 31 C
10:00:01  gateway receives it

-- internet offline --

10:08:15  internet returns
10:08:16  backend receives replay

measured_at      = 10:00:00 UTC
gateway_received_at = 10:00:01 UTC
backend_received_at = 10:08:16 UTC

```

If only backend arrival time were retained, the system would falsely imply that the physical reading occurred at 10:08:16.

4.2 History ordering rule

```

if measured_at is trustworthy:
    use measured_at
else:
    use gateway_received_at

```

The quality contract should explicitly distinguish reading quality from clock quality so historical charts and later live-alert processing can decide whether a device timestamp is trustworthy.

```

{
  "quality": {
    "reading": "valid",
    "clock": "synchronised"
  }
}

```

5. Partial Batch Outcomes and Transaction Rules

Mixed batches are intentional. One invalid device event must not block unrelated valid events indefinitely. The API therefore needs per-item outcomes rather than a single opaque batch result.

5.1 Proposed endpoint

```
POST /api/v1/telemetry/batches
```

5.2 Outcome vocabulary

- **accepted** - new valid event persisted successfully.
- **duplicate** - matching identity/content already exists; safe to remove from delivery queue.
- **rejected** - permanent item-level failure with a structured reason code.

Suggested permanent rejection codes for the first contract version:

```
unknown_device
wrong_gateway
unsupported_schema_version
invalid_metric
invalid_unit
value_out_of_range
malformed_value
identity_conflict
```

5.3 Commit boundary

Non-negotiable transaction rule

Never report an item as accepted before the PostgreSQL transaction has committed. If the commit outcome is uncertain to the client, the gateway retries and idempotency protects the database.

```
Receive batch
  |
  v
Schema checks
  |
  v
Ownership checks
  |
  v
Classify permanent rejection
  |
  v
Detect duplicates/conflicts
  |
  v
Insert valid events
  |
  v
COMMIT DATABASE
  |
  v
return item results
```

If PostgreSQL fails before commit, the server must not claim accepted outcomes. Return a transient HTTP failure and allow the gateway to retry.

6. Gateway Queue Lifecycle

The gateway implementation is largely a later reliability milestone, but Sprint 01 must freeze the response semantics it will consume.

```
accepted
  -> delete from gateway queue

duplicate
  -> delete from gateway queue

rejected + permanent reason
  -> move to bounded quarantine
  -> remove from delivery queue

HTTP timeout / 429 / 5xx
  -> keep queued
  -> retry later
```

Responsibility boundary

Backend decides event outcome. Gateway decides queue lifecycle from that outcome.

7. Alert Ordering and State Model

The full alert engine is not a Sprint 01 deliverable, but its event-ordering assumptions must be settled now because telemetry storage and replay semantics will otherwise become coupled to implementation details later.

7.1 Existing alert behavior to preserve

- Three consecutive valid readings above the threshold create one active alert.
- Three consecutive valid readings below the recovery threshold resolve the condition.
- A gap longer than 15 seconds resets a pending consecutive-reading count.
- Invalid readings do not count toward trigger or recovery.
- Readings more than 30 seconds old when received by the backend are stored as history but do not change live temperature alerts.
- Duplicate and out-of-order readings do not advance live alert evaluation.
- Missing telemetry changes connection status; it does not prove temperature recovery.

7.2 Proposed effective ordering rule

```
effective_event_time =
  measured_at, when clock quality is trustworthy
  otherwise gateway_received_at

event_time <= last_alert_evaluated_event_time
=> historical/out-of-order
=> store event
=> DO NOT advance alert state

backend_received_at - effective_event_time > 30 seconds
=> historical
=> DO NOT advance live alert state
```

7.3 Mixed ordering example

```
sequence 102    10:00:10    31.2 C
sequence 100    10:00:00    31.1 C
sequence 101    10:00:05    31.5 C
sequence 103    10:00:15    31.7 C
```

The backend should not evaluate live alert state using JSON array order. Valid alert candidates should be evaluated according to the agreed effective event chronology, while duplicates and stale events are excluded from advancing the state machine.

7.4 Reboot boundary

Sequence number is monotonic only within a boot. Therefore sequence number alone cannot establish global recency across device reboots.

```
boot A  sequence 5000
boot B  sequence 1    <- may be newer
```

7.5 Alert state model

```

NORMAL
  | high #1
  v
HIGH_PENDING
  | high #2
  v
HIGH_PENDING
  | high #3
  v
ACTIVE
  | recovery #1
  v
RECOVERY_PENDING
  | recovery #2
  v
RECOVERY_PENDING
  | recovery #3
  v
RESOLVED

```

Acknowledgement is intentionally separate from condition state. The system must allow ACTIVE + ACKNOWLEDGED and RESOLVED + UNACKNOWLEDGED combinations, because acknowledgement records operator action but does not itself resolve the physical temperature condition.

8. Site and Authorization Boundary

Decision

Telemetry payload data does not decide or authorize site access. Site membership is derived from the registered gateway/device relationship in backend data.

```

gateway credential
  |
  v
registered gateway
  |
  v
registered device
  |
  v
device.site_id

```

The backend derives gateway identity from its credential and verifies device ownership. A caller-supplied site identifier must never be accepted as proof that the caller is authorized to submit telemetry for that site.

Technical recommendation: omit site_id from the telemetry event body for the MVP. If a future protocol includes it for convenience, treat it as informational rather than authoritative.

9. Technical Decision Register

ID	Decision	Status
D-001	Event identity is (device_id, boot_id, sequence_number).	Accepted
D-002	Database enforces unique composite event identity.	Accepted
D-003	Same identity + same immutable content => duplicate.	Accepted
D-004	Same identity + changed immutable content => reject and log conflict.	Accepted
D-005	Backend assigns backend_received_at; client cannot supply it.	Accepted
D-006	Gateway assigns gateway_received_at on original receipt.	Accepted
D-007	All persisted absolute timestamps use UTC.	Accepted
D-008	Mixed batches produce item-level outcomes.	Accepted

ID	Decision	Status
D-009	Accepted response is emitted only after database commit.	Accepted
D-010	Accepted and matching duplicate events can leave gateway queue.	Accepted
D-011	Permanently invalid events move to bounded quarantine.	Accepted
D-012	Transient failures remain queued for retry.	Accepted
D-013	Duplicates never advance alert state.	Accepted
D-014	Historical/out-of-order events never advance live alert state.	Accepted
D-015	Site is resolved from device registry; telemetry never authorizes itself.	Accepted
D-016	Freeze alert ordering contract in Sprint 01; implement full alert engine later.	Proposed

The released MVP is expected to contain versioned API/telemetry contracts and architectural decision records. Sprint 01 should establish the first decision log so later implementation changes can be traced rather than silently changing contract semantics.

10. Open Questions

The following questions should be resolved or explicitly deferred before Sprint 01 closes. They are intentionally recorded rather than hidden inside implementation code.

#	Question area	Open question	Technical Lead recommendation
1	Clock trust model	What exactly makes measured_at trustworthy? Is clock quality only synchronised/unsynchronised, or should it include NTP state or uncertainty?	Recommend an explicit clock-quality enum in the telemetry contract.
2	Live event ordering	What exact field orders live alert candidates?	Recommend trustworthy measured_at, otherwise gateway_received_at.
3	Identity conflict detection	Compare immutable columns directly or store a canonical content hash?	Either is valid; choose one deterministic implementation and test it.
4	Gateway timestamp trust	Can gateway_received_at be accepted as authoritative for MVP replay semantics?	Document the trust assumption; strengthen later if needed.
5	Rule scope	Is a temperature rule global, per site, per device, or per device class?	Requires Product Owner decision before full alert implementation.
6	Reboot during pending alert	Should a device reboot reset a pending consecutive-reading count?	Recommend reset because reboot is a continuity boundary.
7	Batch ordering	Should alert candidates inside one HTTP batch be reordered by effective event time?	Recommend yes; payload array order should not define physical chronology.
8	Quarantine bound	What size/retention should the invalid-event quarantine use?	Requirements say bounded but do not define the exact limit.
9	Pre-commit failure response	What response is returned if validation is complete but DB commit fails?	Recommend transient request failure and no accepted claims.
10	Concurrent same-device batches	Can multiple batches for one device be processed concurrently?	Need serialization/locking/idempotent state update strategy before alert engine work.

11. Sprint Exit Criteria and Definition of Done

Sprint 01 success condition

The simulator can submit a mixed batch containing new, duplicate, invalid, and out-of-order events, and the backend/database produce deterministic, tested outcomes without requiring later contract redesign.

11.1 Proposed Sprint 01 exit checklist

- ☐ Telemetry schema is versioned and documented.
- ☐ Event identity and immutable-content rules are implemented and tested.
- ☐ PostgreSQL migration enforces event uniqueness.
- ☐ Batch ingestion returns item-level accepted / duplicate / rejected outcomes.
- ☐ Accepted outcomes are returned only after transaction commit.
- ☐ Permanent rejection reasons are structured and documented.
- ☐ Timestamps have explicit ownership and UTC storage semantics.
- ☐ Historical replay cannot overwrite a newer current reading.
- ☐ Duplicate and out-of-order rules for future alert evaluation are documented.
- ☐ Simulator test fixtures cover valid, invalid, duplicate, identity-conflict, stale, and out-of-order telemetry.
- ☐ Decision register and open questions are committed to project documentation.
- ☐ No blocking defect remains for the M1 vertical slice.

11.2 Definition of done alignment

For each Sprint 01 requirement/task, completion should include a linked implementation issue, automated or documented test evidence, verified error behavior, updated API/operational documentation, and no blocking defect. These items align with the project-wide Definition of Done.

Appendix A. API Contract Examples

A.1 Example request

```
{
  "schema_version": 1,
  "events": [
    {
      "device_id": "motor-001",
      "boot_id": "boot-a",
      "sequence_number": 100,
      "measured_at": "2026-09-15T00:30:00Z",
      "device_uptime_ms": 534000,
      "metric": "temperature",
      "value": 31.4,
      "unit": "celsius",
      "quality": {
        "reading": "valid",
        "clock": "synchronised"
      }
    }
  ]
}
```

A.2 Example mixed-batch response

```
{
  "batch_id": "01K5EXAMPLE",
  "results": [
    {
      "device_id": "motor-001",
      "boot_id": "boot-a",
      "sequence_number": 100,
      "outcome": "accepted"
    },
    {
      "device_id": "motor-001",
      "boot_id": "boot-a",
      "sequence_number": 101,
      "outcome": "duplicate"
    },
    {
      "device_id": "unknown-99",
      "boot_id": "boot-c",
      "sequence_number": 12,
      "outcome": "rejected",
      "reason": "unknown_device"
    }
  ]
}
```

A.3 Requirement traceability used in this report

Requirement / milestone	Relevance to Sprint 01 technical contract
M1	Contracts and vertical slice: requirements, telemetry schema, database migrations, simulator-to-API ingestion.
FR-05	Every reading has stable event identity.
FR-08	Ingestion is idempotent; conflicting reuse of identity is rejected and logged.
FR-09	Each batch item is validated; valid items in a mixed batch can still succeed.
FR-11	Historical charts use trustworthy measurement time and do not imply continuity across gaps.
FR-14 / Alert behavior	Consecutive-reading alert episodes, stale-event rules, duplicate/out-of-order exclusion.
AT-05	Retry after backend commit but interrupted response must remain safe.
AT-06	Mixed valid, invalid, and duplicate readings receive correct outcomes.

End of report